

Metanoia: recording cue sheet

These are talking points, not a paragraph to read word for word. **Bold blue** is the line to actually say. The rest is what to point at. The handwritten lines are the ones that land, so say them like you mean them.

1 What it is

0:00 · 30s

- Metanoia is a storefront for **agent-led subscription commerce**, built on Juspay Hyperswitch.
- *"You give an AI agent a spending mandate instead of your credit card. It shops, compares, and buys a subscription for you, and Hyperswitch settles the real payment in the sandbox."*
- **The whole design in one line: the model proposes, the server decides, Hyperswitch settles.** An LLM can recommend. It can never set a price or move money.

2 The live demo (deployed URL)

0:30 · 3 min

- **Home and mandate.** Point at the three limits: \$60 a month, \$40 per charge, a max number of subscriptions. **"I set the envelope. The server enforces it. The agent cannot touch these."**
- **Context.** "I can give it optional signals." Show the GitHub repo (aftertalk) and the X profile you typed in. Then type the request: "LLM inference API, tool calling and long context, under \$40."
- **Run, then Decision Authority.** **"Gemini proposed Apex LLM Pro. The server overrode it and chose Relay LLM, because it scored higher on my priorities, 76 to 71."** Point at MODEL PROPOSED versus SERVER FINAL, and the red SERVER OVERRIDE reason.
- **Point at the deterministic ranking bars,** SpendGuard 4 of 4 PASS, and Nano Chat **blocked** for missing tool calling. **"The scores are computed, not the model's opinion."**
- **Confirm, then Checkout.** **"This is real Hyperswitch Unified Checkout. I type the test card into their iframe, so the card never touches my server."** Card 4242 4242 4242 4242, then pay.
- **Receipt.** "Settled." Point at capability 200 · SANDBOX LIVE, which proves what it bought works, and the green **Signed webhook delivered** line on the trace (refresh if the async event is still in flight). Then scroll to the **live payment record.** **"This is pulled straight from the Hyperswitch API: connector, transaction ID, card network. The same record their dashboard shows. Nothing faked."**
- **Honesty beats.** Over-budget A100 gives a big red **Denied**, "card never touched." Then "find cheaper" on the LLM gives a **no-match tradeoffs** screen with no charge. **"A budget refusal and 'we don't carry that' are different screens. I never blur them."**
- **Extras.** Subscriptions tab (view and cancel), Payment Test Lab (test cards and a real owned-only refund), and session isolation. **"Open my receipt from another browser and you get nothing."**

3 The GitHub repo

3:30 · 1 min

- github.com/theaayushsthal/metanoia. **"Everything is here, and it is honest about what is proven versus deferred."**
- **Point at the layout.** `app/` holds the routes, workbench, and checkout. `lib/agent/` holds procure, ranking, spendCap, and scouts. `lib/checkout` and `lib/hyperswitch` are the only code that can move money, and the agent cannot reach them. `lib/ap2/` holds the mandate shapes, and `lib/db` plus store is the dual backend.
- **"87 tests green, type-check and lint clean."** Call out the tests that matter: SpendGuard, the decision-authority override, idempotency, renewal, and session ownership.
- **"It is deployed on Cloud Run, Cloud SQL, Vertex, and Secret Manager, and a real payment completes on the live URL."**

4 The architecture doc (the PDF)

4:30 · 1 min

- "I framed it around the industry and the payment choices, in three pages." Walk the three figures.
- Fig 1, the end-to-end flow. "The green boxes are the deterministic authority. The red dashed ones are deferred or refused, where the card is never touched."
- Fig 2, the authority boundary. "Model proposes, server decides, Hyperswitch settles. Only a proposal crosses from the model to the server. Only the server can move money. The LLM has no payment tool."
- Fig 3, the payment sequence. "SpendGuard runs again on the server before Hyperswitch is ever called. A refusal means a 403 and no payment intent."
- Mention the trust boundaries: untrusted input wrapped as "data, not instructions," an SSRF-guarded GitHub fetch, and the secret key kept server-side only.

5 The decisions doc (ADRs)

5:30 · 1 min

- "Every real decision is written down, with the alternative I rejected." The payment ones I would highlight:
- Model proposes, server decides. The LLM only emits a structured proposal. `decide()` re-validates on server-owned prices and can override it.
- SpendGuard before any charge. It checks the caps in order and returns 403 before Hyperswitch is contacted.
- Fauxpay for checkout, Stripe reserved for recurring. "Fauxpay reliably settles the checkout. Real off-session recurring needs a reusable payment-method id, which only a real connector returns."
- Stable payment IDs give idempotency. `pay_ = sha256(customer:plan:period)`, so a retry reuses the intent and never charges twice.
- Pending-first renewal for crash durability, and atomic, HMAC-verified webhooks that retain unknown events.
- "Connectors are processors, not payment methods. The customer picks a card, and routing is my decision. Unified Checkout keeps me out of PCI scope."

6 Webhooks and the honest limits

6:30 · 30s

- Webhooks (proven). "Signed Hyperswitch events reach an HMAC-verifying ingress, which forwards the untouched body to the Cloud Run receiver. Cloud Run verifies the HMAC again and settles the payment in Cloud SQL, deduped by event id." I confirmed it in the ingress and Cloud Run logs and the events table, for real payments. The low-level reason a direct `run.app` delivery does not land is still unconfirmed.
- Off-session MIT (the one honest edge). It is coded, but connector-blocked: Stripe returns `UE_9000` until the account gets raw-card API access. Fauxpay returns no reusable method.
- Deferred. The automatic renewal scheduler, where the path is already tested, and AP2 cryptographic mandate signatures.

7 Close

7:00 · 15s

- "Agents are becoming the customer. Metanoia is the settlement layer for when an agent actually buys, on Hyperswitch."
- "And the one guarantee under all of it: the number that gets charged is computed by deterministic, unit-tested server code, on data the model never authored."

Before you record: have the deployed URL, the GitHub repo, and both PDFs open in tabs. Payments occasionally 429 on the sandbox, so if the Pay button stalls, click it once more. Keep the mandate visible when you say "the server enforces it."